Data Communications & Networks

Chapter 9: Sections 9.1, 9.2.1 & 9.2.2

Introduction to Data-Link Layer & Link-Layer Addressing (ARP)

Topics Covered:

- 9.1 Introduction: Nodes, Links, Services, Sublayers
- 9.2.1 Link-Layer Addresses (MAC Addresses)
- 9.2.2 ARP: Address Resolution Protocol
  (ARP Request, Reply, Packet Format, & Packet Journey)

Reference: Forouzan, "Data Communications and Networking," 5th ed.

April 11, 2026

Contents

1 Introduction to Data-Link Layer — Section 9.1 **2**
1.1 Nodes and Links — Section 9.1.1 2
1.2 Services of the Data-Link Layer — Section 9.1.2 3
1.2.1 1. Framing 3
1.2.2 2. Flow Control 4
1.2.3 3. Error Control 4
1.2.4 4. Congestion Control 4
1.3 Two Categories of Links — Section 9.1.3 4
1.4 Two Sublayers — Section 9.1.4 5

2 Link-Layer Addressing — Section 9.2 **6**
2.1 Types of Link-Layer Addresses — Section 9.2.1 6
2.1.1 Unicast, Multicast, and Broadcast MAC Addresses 7
2.1.2 Why IP Addresses Remain the Same but MAC Addresses Change 8
2.2 Address Resolution Protocol (ARP) — Section 9.2.2 8
2.2.1 ARP Request and Reply 8
2.2.2 ARP Packet Format 9
2.2.3 Complete Packet Journey: Alice to Bob (Full Example) 10
2.2.4 ARP Caching 12

Chapter 1 Introduction to Data-Link Layer — Section 9.1

What Is the Data-Link Layer?

You already know the network layer (IP addresses, routing) gets a packet from source host to destination host. But how does the packet actually travel across each individual link between routers? That's the job of the **data-link layer**.
The data-link layer's domain:

- While the network layer handles the *end-to-end* journey (source host to destination host),
- the data-link layer handles *one hop at a time* — from one node to the very next node.
- "One hop" = one physical link (LAN cable, Wi-Fi, fiber, etc.).

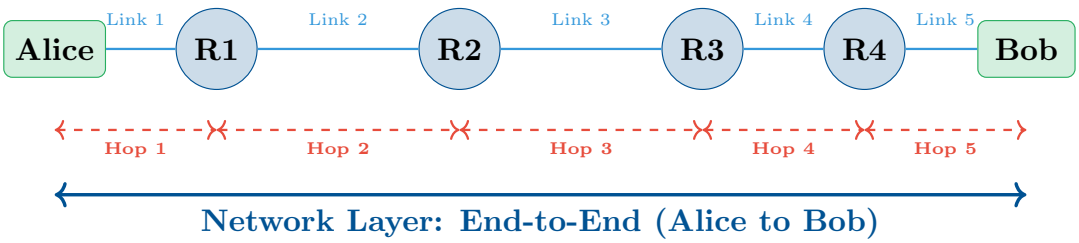
An analogy: Think of traveling from Lahore to London. The entire journey is the network layer's job. But the data-link layer only cares about one leg at a time: Lahore to Dubai airport (flight), then Dubai airport to London Heathrow (another flight). Each leg is one "hop."

1.1 Nodes and Links — Section 9.1.1

Nodes and Links Defined

In the context of the data-link layer:

- **Nodes** = hosts (computers, servers) and routers. These are the endpoints and intermediate processing points.
- **Links** = the physical networks connecting nodes. A LAN is a link, a WAN connection is a link.
- The data-link layer's job: deliver a frame from one node to the **adjacent** node on the same link.



Each data-link layer hop delivers a frame to the very next node

1.2 Services of the Data-Link Layer — Section 9.1.2

Four Services Provided by the Data-Link Layer

The data-link layer provides the following services to the network layer above it:

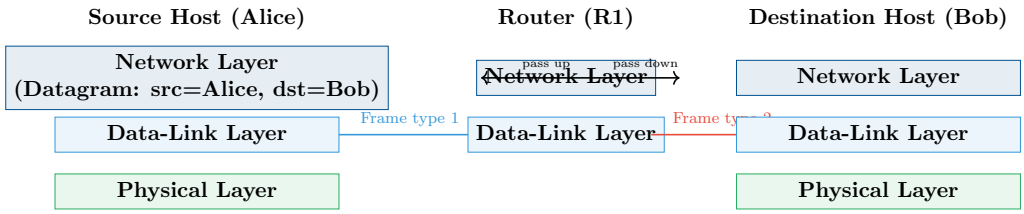
1.2.1 1. Framing

Framing: Packaging the Datagram

The data-link layer wraps the network-layer **datagram (packet)** in a **frame**.

- A frame = datagram + header (and sometimes a trailer).
- The header contains link-layer (MAC) addresses: destination and source.
- A frame is the unit of transmission at the data-link layer.
- Different links use different frame formats.

At the receiving node, the data-link layer **decapsulates** the datagram from the frame and passes it up to the network layer.
Why re-encapsulate at every router? Because different links may use completely different protocols and frame formats (Ethernet on Link 1, Wi-Fi on Link 2, fiber on Link 3). Each requires its own frame format!



Re-encapsulation at each router: same datagram, different frame each time

1.2.2 2. Flow Control

Flow Control: Matching Speeds

If the sender produces frames faster than the receiver can consume them:

- Buffer overflow at the receiver.
- Two solutions: **drop excess frames** or **feedback** to slow down the sender.
- The data-link layer can implement flow control (though TCP at transport layer also does this for end-to-end).

1.2.3 3. Error Control

Error Control: Detecting and Fixing Errors

Electromagnetic signals are susceptible to interference, attenuation, and noise. These cause bit errors in frames. The data-link layer must:

1. **Detect** errors (using checksums, CRC — studied in Chapter 10).
2. **Correct** errors at the receiver (forward error correction), OR
3. **Discard** corrupted frames and ask for retransmission.

Error control at the data-link layer is *per-link*. The transport layer (TCP) handles end-to-end reliability.

1.2.4 4. Congestion Control

Most data-link layer protocols do **not** directly handle congestion (that's the network and transport layers' job). However, some wide-area networks (e.g., Frame Relay) do implement basic congestion signaling.

1.3 Two Categories of Links — Section 9.1.3

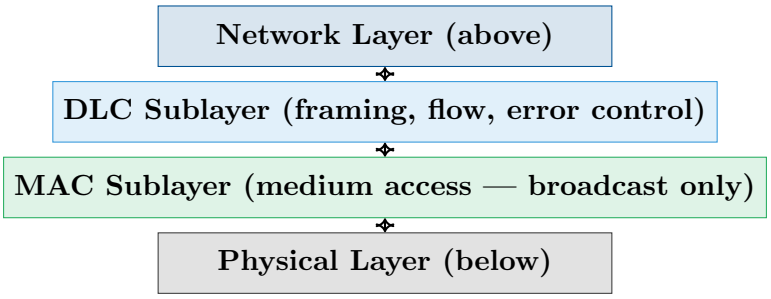
Feature	Point-to-Point Link	Broadcast Link
Shared?	No (dedicated to 2 devices)	Yes (shared among many)
Example	Fiber between two routers	Ethernet LAN, Wi-Fi
MAC needed?	Not always	Yes (multiple access control)
Who receives?	Only the connected device	All devices on the link

1.4 Two Sublayers — Section 9.1.4

DLC and MAC Sublayers

The data-link layer is divided into two sublayers:

1. **Data Link Control (DLC)**: Handles issues common to both point-to-point and broadcast links. Covers framing, flow control, and error control.
2. **Media Access Control (MAC)**: Handles issues specific to *broadcast* links — how multiple devices share a shared medium without collisions. This is what Ethernet (CSMA/CD), Wi-Fi (CSMA/CA), etc. address.



Chapter 2 Link-Layer Addressing — Section 9.2

Why Do We Need Link-Layer Addresses?

You might ask: we already have IP addresses (network layer). Why do we need *another* set of addresses at the data-link layer?

The problem:

- IP addresses define the *end-to-end* source and destination. These do NOT change as the packet travels through routers.
- But as a packet hops through routers, each hop uses a *different physical link*.
- Each link delivers the frame to the next router's specific interface — not the final destination.
- The IP address of Router R1 doesn't tell you *which wire* on that router's Ethernet card to send to!
- We need **link-layer addresses** (also called **MAC addresses** or **physical addresses**) to identify the specific hardware interface for each hop.

Rule:

- IP (network-layer) addresses: **stay the same** from source to destination. They are the "passport" of the packet.
- MAC (link-layer) addresses: **change at every hop**. They identify the next immediate destination on each link, like boarding passes that change at each airport.

2.1 Types of Link-Layer Addresses — Section 9.2.1

2.1.1 Unicast, Multicast, and Broadcast MAC Addresses

Three Types of Link-Layer Addresses

1. **Unicast address**: Identifies a *single specific* network interface (one device). Every NIC (Network Interface Card) has a globally unique unicast MAC address. Example: `3A:1B:4C:2D:5E:6F`. The LSB (least significant bit) of the first byte is 0.

2. **Multicast address**: A special address that a *group* of interfaces can belong to. A packet with a multicast destination is delivered to all members of that group on the link. The LSB of the first byte is 1 (to distinguish from unicast). Example: `01:00:5E:xx:xx:xx` (for IPv4 multicast on Ethernet).

3. **Broadcast address**: The "send to everyone" address. A frame with this destination is delivered to *all* devices on the link. Ethernet broadcast: `FF:FF:FF:FF:FF:FF` (all 48 bits = 1).

MAC Address Structure

bit 0 = I/G (Unicast/Multicast) bit 1 = U/L (Universal/Local)

OUI (24 bits): Manufacturer ID	Serial Number (24 bits): Device ID

← 48 bits total = 6 bytes = 12 hex digits (e.g., AA:BB:CC:DD:EE:FF)

Example 9.1: Unicast MAC Address

Question: In Ethernet (most common LAN), link-layer addresses are 48 bits long, expressed as 12 hexadecimal digits. Is the address `3A:1B:4C:2D:5E:6F` a unicast, multicast, or broadcast address?

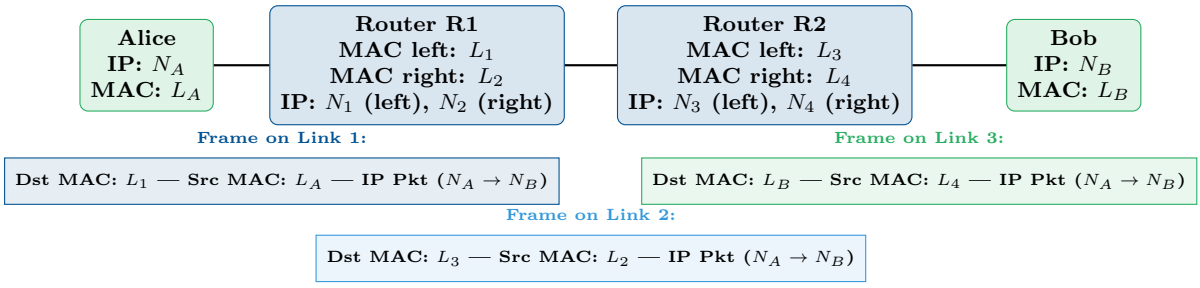
Solution:

- First byte in hex: `3A` = 0011 1010 in binary.
- Least significant bit (LSB) = 0 ⇒ this is a **unicast** address.

Example 9.2: Ethernet Multicast

Multicast addresses in Ethernet have an *odd first byte*. For example: `01:00:5E:XX:XX:XX` (first byte = `01` = 0000 0001; LSB = 1 ⇒ multicast).
Devices that want to receive multicast traffic must register their network interface to "listen" on the multicast MAC address.

2.1.2 Why IP Addresses Remain the Same but MAC Addresses Change



IP addresses ($N_A \to N_B$): UNCHANGED throughout the journey.
MAC addresses: CHANGE at every hop (destination MAC = next-hop MAC).

2.2 Address Resolution Protocol (ARP) — Section 9.2.2

The Problem ARP Solves

When a node (host or router) needs to send a frame to the **next hop**, it knows:

- The **IP address** of the next hop (from the routing/forwarding table).
- But **NOT the MAC address** of the next hop!

To create the frame, it needs the destination MAC address.
ARP (Address Resolution Protocol) is the mechanism that finds the MAC address corresponding to a known IP address.
ARP Slogan: "I know your IP address. What is your MAC address?"

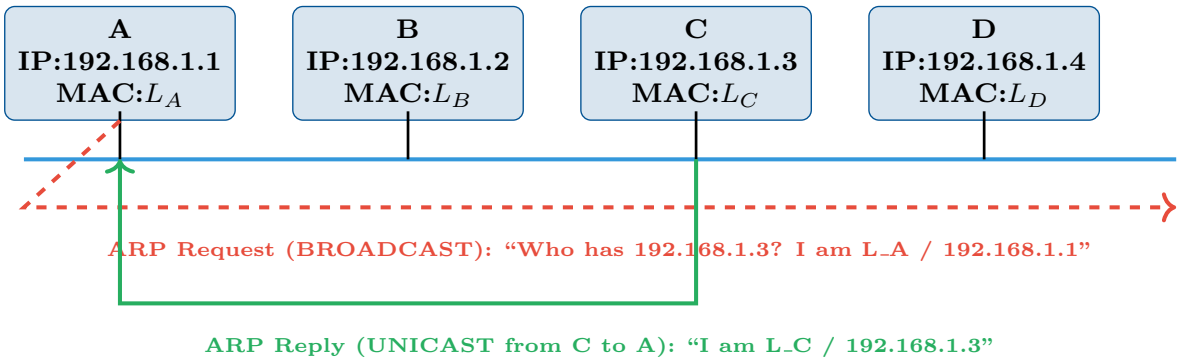
2.2.1 ARP Request and Reply

How ARP Works: Step by Step

1. **ARP Request (Broadcast):**
   - The asking node broadcasts an ARP request frame to all devices on the link.
   - Destination MAC in the frame = `FF:FF:FF:FF:FF:FF` (broadcast).
   - The ARP packet contains: "I am MAC L_A, IP N_A. Who has IP N_X? Please tell me your MAC."
   - **Every device** on the link receives this broadcast and checks if the target IP matches their own.
2. **ARP Reply (Unicast):**
   - Only the device with IP N_X responds.

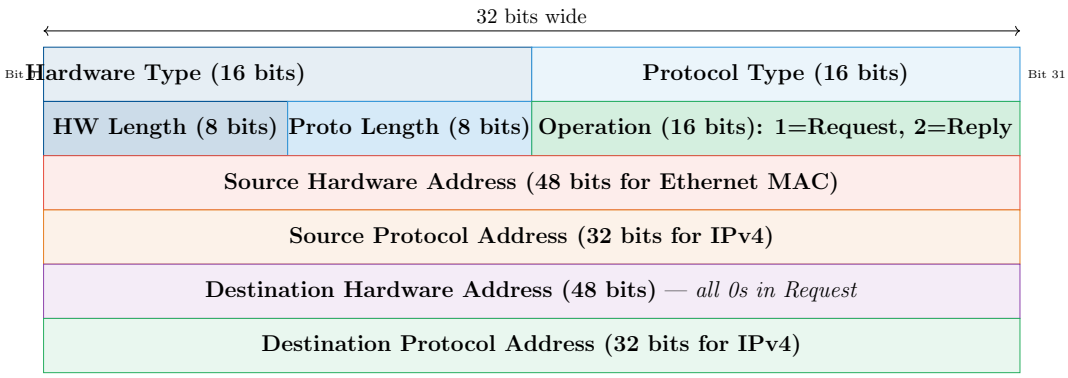
- It sends an **ARP reply** directly back (unicast) to L_A.
- The reply says: "I am MAC L_X, IP N_X."

3. **Caching:**
   - The asking node stores the mapping (IP → MAC) in its **ARP cache** (table) for future use.
   - Cache entries expire after a timeout (typically 20 minutes) to handle network changes.
   - This avoids broadcasting an ARP request every time a frame needs to be sent.



A broadcasts; all receive but only C replies; A stores L_C in ARP cache

2.2.2 ARP Packet Format



Field	Description
Hardware Type	Type of link layer (1 = Ethernet)
Protocol Type	Type of network layer (0x0800 = IPv4)
Hardware Length	MAC address length in bytes (6 for Ethernet)
Protocol Length	IP address length in bytes (4 for IPv4)
Operation	1 = ARP Request, 2 = ARP Reply
Source Hardware Addr	Sender's MAC address
Source Protocol Addr	Sender's IP address
Dest. Hardware Addr	Target MAC (all 0s in Request because unknown!)
Dest. Protocol Addr	Target IP address

Example 9.3: ARP Request Packet

Scenario: Host A (IP: 192.168.1.1, MAC: AA:BB:CC:11:22:33) wants to find the MAC address of Host B (IP: 192.168.1.5).

ARP Request packet contents:

- Hardware Type: 1 (Ethernet)
- Protocol Type: 0x0800 (IPv4)
- Operation: 1 (Request)
- Source Hardware: `AA:BB:CC:11:22:33` (A's MAC)
- Source Protocol: `192.168.1.1` (A's IP)
- Destination Hardware: `00:00:00:00:00:00` (UNKNOWN - that's what we're asking!)
- Destination Protocol: `192.168.1.5` (B's IP - this is what we know)

This packet is placed in a frame with destination MAC = `FF:FF:FF:FF:FF:FF` (broadcast).

Example 9.4: ARP Reply Packet

Scenario: Host B (IP: 192.168.1.5, MAC: DD:EE:FF:44:55:66) receives the ARP request and replies.

ARP Reply packet (from B to A):

- Operation: 2 (Reply)
- Source Hardware: `DD:EE:FF:44:55:66` (B's MAC — this is the answer!)
- Source Protocol: `192.168.1.5` (B's IP)
- Destination Hardware: `AA:BB:CC:11:22:33` (A's MAC)
- Destination Protocol: `192.168.1.1` (A's IP)

This reply is a unicast frame (sent only to A's MAC). Host A receives it and stores: `192.168.1.5` → `DD:EE:FF:44:55:66` in its ARP cache.

2.2.3 Complete Packet Journey: Alice to Bob (Full Example)

Let's walk through the **complete journey** of a packet from Alice to Bob, seeing how IP addresses stay the same but MAC addresses change at every hop.

Scenario Setup

- Alice's host: IP = N_A, MAC = L_A
- Router R1: Left interface IP = N_1, MAC = L_1; Right interface IP = N_2, MAC = L_2
- Router R2: Left interface IP = N_3, MAC = L_3; Right interface IP = N_4, MAC = L_4
- Bob's host: IP = N_B, MAC = L_B
- Links: Alice $\xrightarrow{\text{Link 1}}$ R1 $\xrightarrow{\text{Link 2}}$ R2 $\xrightarrow{\text{Link 3}}$ Bob

Step 1: Activities at Alice's site

1. Alice's **application layer** prepares a message for Bob.
2. It passes down to the **network layer**, which creates a datagram with:
   - Source IP = N_A, Destination IP = N_B (these never change!).
3. The network layer checks its **forwarding table**: to reach N_B, the next hop is Router R1 with IP N_1.
4. Alice runs **ARP**: "What is the MAC of N_1?" $\Rightarrow$ gets back L_1.
5. The network layer passes the datagram + destination MAC L_1 to the data-link layer.
6. The data-link layer creates **Frame on Link 1**:
$$\text{Dst MAC: } L_1 \mid \text{Src MAC: } L_A \mid \text{IP Datagram}(N_A \to N_B)$$
7. The physical layer converts to signals and sends on Link 1.

Step 2: Activities at Router R1

1. Physical layer receives signals, converts to bits, creates the frame.
2. **Data-link layer** decapsulates the frame (sees Dst MAC = L_1 = my MAC ✓), extracts the datagram.
3. Passes datagram up to the **network layer**.
4. Network layer reads: Destination IP = N_B. Consults forwarding table: next hop is R2 with IP N_3.
5. Runs **ARP**: "What is MAC of N_3?" $\Rightarrow$ gets L_3.
6. Passes datagram + destination L_3 **down to data-link layer of the right interface**.
7. Data-link creates **Frame on Link 2**:
$$\text{Dst MAC: } L_3 \mid \text{Src MAC: } L_2 \mid \text{IP Datagram}(N_A \to N_B)$$
8. Physical layer sends on Link 2.

Step 3: Activities at Router R2

Almost identical to R1:

1. Receives frame on Link 2. Decapsulates. Network layer reads destination = N_B.
2. Forwarding table says: N_B is directly connected (on Link 3). Next hop = N_B.
3. Runs **ARP**: "What is MAC of N_B?" $\Rightarrow$ gets L_B.
4. Creates **Frame on Link 3**:
$$\text{Dst MAC: } L_B \mid \text{Src MAC: } L_4 \mid \text{IP Datagram}(N_A \to N_B)$$
5. Sends on Link 3.

Step 4: Activities at Bob's site

1. Physical layer receives signals, converts to frame.
2. Data-link layer decapsulates (Dst MAC = L_B = my MAC ✓), extracts datagram.
3. Passes to network layer, which reads: Destination IP = N_B = *that's me!*
4. Passes the data up through transport layer to the application. Bob receives Alice's message!

Summary of Address Changes During Alice → Bob Journey

Frame on Link	Dst MAC (changes)	Src MAC (changes)	IP Src → Dst (FIXED)
Link 1 (Alice → R1)	L_1 (R1's left MAC)	L_A (Alice's MAC)	$N_A \to N_B$
Link 2 (R1 → R2)	L_3 (R2's left MAC)	L_2 (R1's right MAC)	$N_A \to N_B$
Link 3 (R2 → Bob)	L_B (Bob's MAC)	L_4 (R2's right MAC)	$N_A \to N_B$

IP addresses ($N_A \to N_B$): **NEVER CHANGE**. MAC addresses: **CHANGE AT EVERY HOP.**

Why don't we put all the intermediate router MACs in the original frame?
Because we don't know the path in advance! The Internet uses dynamic routing — the path can change. Also, each link uses a different protocol. The frame is rebuilt at every router with fresh MAC addresses for the next hop only.

2.2.4 ARP Caching

ARP Cache: Remembering Recent Answers

Running an ARP request for every single frame would create enormous broadcast traffic. **ARP caching** solves this:

- After each successful ARP exchange, the mapping (IP → MAC) is stored in an **ARP cache** (also called ARP table).
- Future frames to the same IP use the cached MAC immediately — no broadcast needed!
- Entries **expire** after a timeout (e.g., 20 minutes) to handle changes (NIC replaced, IP reassigned).
- When a node receives any ARP packet (request or reply), it caches the *sender's* mapping even without being the target (opportunity caching).

You can view your ARP cache: `arp -a` on Windows/Linux!

Chapter Summary

Key Takeaways — Chapter 9

Section 9.1 — Data-Link Layer Introduction:

- Communication at data-link layer is **node-to-node** (one hop at a time).
- **Nodes** = hosts + routers; **Links** = physical networks between nodes.
- Four services: **Framing**, **Flow Control**, **Error Control**, **Congestion Control**.
- Datagram is **re-encapsulated** into a new frame at each router (different frame format per link).
- Two link types: **Point-to-point** (dedicated) and **Broadcast** (shared).
- Two sublayers: **DLC** (common to all) and **MAC** (for broadcast links only).

Section 9.2.1 — Link-Layer Addressing:

- Link-layer addresses (MAC/physical addresses) are **48 bits** (6 bytes) in Ethernet.
- **Unicast**: LSB of first byte = 0. **Multicast**: LSB = 1. **Broadcast**: all bits = 1 (`FF:FF:FF:FF:FF:FF`).
- IP addresses NEVER change end-to-end. MAC addresses CHANGE at every hop.

Section 9.2.2 — ARP:

- ARP maps IP address → MAC address for the next hop.
- ARP Request = **broadcast** (all devices receive it). ARP Reply = **unicast** (only the requester).
- ARP packet: Hardware type, Protocol type, HW/Proto lengths, Operation, src/dst HW+proto addresses.
- ARP cache stores mappings to avoid repeated broadcasts. Expires after timeout.
- At each router: decapsulate frame, network layer looks up next hop IP, ARP finds next hop MAC, create new frame with new MACs.